

PLAIN-LANGUAGE RESEARCH PROPOSAL

DeepAlign-Bench | 完整人话版

把绝对适配、反事实特异性和非补偿评分逐步说清楚

适合组内共识 · 导师讨论 · 正式写作前校验

文档版本	v0.55 · 完整人话版
日期	2026 年 8 月 17 日
研究主线	什么该变 · 两边是否变对 · 是否真增益 · 哪些不能补偿

核心命题

报告对一个人看起来不错，不等于系统会随着用户变化而双向正确改变。

内容导航

1. PDR 与 DeepAlign 的区别
2. Case / Task / User 元数据
3. Persona 与渠道构造
4. 输出条件与 Rubric
5. 非补偿评分
6. family-level 统计
7. 最小实验
8. 逐周执行与论文主张
9. 参考文献

阅读方式：先看主图和研究概要；第 1-7 节解释数据、渠道、rubric、指标和统计；第 8-10 节给出实验结论与投稿决策。

研究概要

一句话说明

我们不是只问“这份报告、代码 patch 或数据分析对某个人看起来合不合适”，而是把同一个任务和同一份证据/仓库/数据交给两位需求不同但都合理用户，看 agent 是否真的会随着目标用户改变最终交付，而且两边都改对。当前只实例化三个代表性场景：网络研究、仓库级软件工程和数据分析，不声称包含所有知识工作。

为什么这和 PDR-Bench 不完全一样

[PDR-Bench](#) 的重要贡献是：把 task 和 persona 一起交给评分器，判断报告的目标、内容、呈现和行动建议是否适合该用户。这个分数有用，但它是单用户绝对评分。

举个例子。同一个团队知识工具选型任务：

- 用户 A 有敏感项目，关键约束是本地部署；
- 用户 B 接受 SaaS，最看重协作方便、维护成本低。

一份“产品比较很完整、表格很好看、建议也像样”的通用报告，对 A 和 B 都可能拿高分。但这不能证明 agent 真正识别了两人的不同。我们要把为 A 写的报告和为 B 写的报告交叉评分：A 的标准是否更喜欢 A 报告；B 的标准是否更喜欢 B 报告；而且两边都必须成立。

这次不再把 clarification 单独变成新题

clarification 已经有很多直接近邻。它在 DeepAlign 里只是一种用户信息进入系统的方式：

模糊但足以开始的 query → agent 判断是否要问 → 用户按隐藏 ledger 回答 → agent 做研究 → final report

它和 structured persona、natural history 使用同一个用户真值和同一套最终报告标准。这样我们能比较：信息直接给出来时会不会用；需要先问出来时还能不能用。我们不声称首次研究 when-to-ask。

DeepAlign-Bench：从 paired users 到反事实特异性

DEEPALIGN-BENCH · EVALUATION GROUND-TRUTH PIPELINE · v0.55

真人真值 → 反事实关系 → 可执行评分

核心对象是 $C(T,E,U_a,U_b)$ ；rubric 是编译产物，judge 只执行冻结标准



最强攻击不是“能不能跑”，而是“这个 gold 和 judge 凭什么可信”



Claim boundary：若无重分类或真人增量效度，只能称 transparent measurement extension

DeepAlign-Bench · 2026-08-17

图1 谁提供真值、谁审计、谁编译、谁评分相互分离；freeze 只负责防止事后改标准。

1. 一个 case 到底由什么组成

1.1 Task family

Task family 不是“采购类”“旅游类”这样的主题标签。它是一份可以生成受控实验条件的蓝图：

- 固定任务核心：所有用户都在解决同一件事；
- 固定 evidence：看到相同资料、日期和候选信息；
- 固定工具、时间、预算和交付格式；
- 两位都可能真实提出这个任务的用户；
- 2-4 个会真正改变结论或行动的用户差异；
- 预先写好的“什么必须变、什么不能变、什么不能做”；
- task-only、matched、swapped 和压力测试报告。

如果换用户后连任务、资料或交付格式都变了，就不知道分数变化来自用户还是来自题目，因此不能做反事实比较。

1.2 Case metadata

Case metadata 记录这一次具体运行是谁、何时、在哪个环境下完成的：

- `case_id / family_id / user_id / channel_id`；
- agent、模型、版本、prompt、随机 seed；
- 搜索后端、工具版本、预算和权限；
- evidence snapshot 与哈希；
- artifact、轨迹、澄清对话和 judge 输出位置；
- 运行状态、失败原因、成本、时间戳。

这层主要是为了复现和排查混杂，程序自动填写，人类抽查。

1.3 Task metadata

Task metadata 记录这道任务在研究上是什么：

- 主要 research intent：比较、规划、选择、诊断、综合还是验证；
- stakes：错误会造成多大损失；
- 交付物：报告、路线图、采购建议、证据表还是行动方案；
- 关键 decision nodes：哪些选择会因用户而改变；
- 共同事实和证据要求；
- 允许工具、时限、信息新鲜度和权限；
- 哪些用户信息缺失时应该澄清。

这层不能全部交给 LLM。两名标注员在看模型输出前独立标注，冲突仲裁。LLM 只负责预填候选和解释理由。

1.4 User-state ledger

Persona 不是写一篇人物小传。真正的核心是一份隐藏的 user-state ledger：

- 事实是什么；
- 谁提供的、何时提供的；
- 可信度和是否可能过期；
- 是否敏感、是否允许用于推理、是否允许写进报告；
- 它会改变哪个 decision node；
- matched 应该怎么处理；
- swapped 为什么不合适；
- 如果不知道，agent 是否应该问。

每条事实如果不能映射到任务决策，就不进入核心 persona，只能作为 irrelevant control。

2. Persona 怎么做得自然

最可靠的顺序是“先给真人一组可能相关的真实任务，再让他选择”，而不是先写两个人设再硬塞任务。

1. 每位参与者从随机化或分层 task slate 里选 3-5 个现实中可能真的要做的任务；保留他看过、认为相关、最后选择和跳过的完整记录。
2. 对每道已选题先让用户自由说目标、限制、风险、取舍、受众、使用方式和“什么会让我不用这个结果”，避免问卷先把答案塞给他。
3. 再用任务类型对应的追问表补漏。每条事实标清是用户主动说的、追问后说的、不适用，还是拒绝回答；也记录不确定性、多个可接受答案、时间戳和隐私许可。
4. 把这些信息整理成用户本人确认的后台 ledger。Agent 默认看不到整个 ledger，只看到实验指定的 persona/history/clarification view。
5. 同一题里不只挑“差异巨大”的两个人，还保留差异小的 near-neighbor pair，以及本来不该导致报告变化的 neutral pair。这样既测“该改时会不会改”，也测“不该改时会不会乱改”。

接着先造的不是两份 rubric，而是一张 **Counterfactual Difference Map (CDM)**。它逐个写清：哪个决策变量应该不同，哪个应该相同，哪些方案对用户来说都可以，什么绝对不能做，以及信息不足时应不应该问。每个节点必须能指回用户确认的事实、共同任务证据或权限规则。

LLM 可以帮忙高召回找候选和拆项，但它没有决定真值的权力。用户本人决定自己的目标和取舍；两名标注员检查来源、是否有决策后果、是否可观察、是否重复、是否带刻板印象；领域专家只管事实、技术可行性和安全。用户不能把错误技术事实说成正确，专家也不能替用户决定“我更在意什么”。

这里有多次 freeze。第一次在写参考答案前冻结 CDM，避免参考答案反过来定义标准；第二次在任何被测 agent 输出前冻结最终 rubric。Freeze 只解决“不能看完答案再改标准”，不证明标准一定对。真实性来自真人确认，遗漏控制来自覆盖审计，最后执行是否可靠要靠 checker、D-JQS 和盲化人评。

3. 先统一建模 Deep Research 怎么和用户、memory 与工作区交互

不能把“一次性完成、主动澄清、中间提问、memory”简单列成四种同类东西。一次性和中间提问说的是什么时候能交互；memory 说的是信息放在哪里以及谁去取；用户中途改预算说的是信息发生了更新。如果只写一个 channel 标签，我们不知道系统为什么变好或变差。

因此，一次实际运行都写成一条 research episode 时间线，至少回答六个白话问题：

1. 用户最开始说的内容，是完整的，还是只够写通用报告，还是不回答就根本做不了？
2. agent 在开始前、搜索中、checkpoint 或草稿后能不能问用户？
3. 信息最初来自当前用户、历史对话、memory、用户档案、邮件/文件、组织规则、行为轨迹还是 agent 自己推断？
4. 信息是直接塞进 prompt、原本可见、agent 主动问到、主动检索到，还是实验环境按时间注入？
5. 信息是当前的、过期的、冲突的，还是中途更新并覆盖旧值？
6. 这个产品本身是否真的支持 ask、memory retrieval、workspace search、checkpoint 和修改草稿？不支持就写“不适用”，不能打零分。

完整库有八种常见范式，但第一批只跑四种：

- P0 task-only closed：没有用户信息，也不能问，得到通用质量 baseline；
- P1 one-shot direct：开始前一次性给完整用户信息，测“给到后会不会用”；
- P2 pre-research clarification：问题本身能开始做，但缺关键个性化信息，测 agent 会不会主动问并真正采用；
- P4 checkpoint update：做到一半收到新预算或新目标，测会不会更新计划、删除旧结论，同时保留没有变化的条件。

研究中提问、memory retrieval、私有工作区和看完草稿再修改先放到扩展条件。我们把“所有范式”定义完整，但不会把所有组合都跑一遍。

3.1 Structured persona

直接给字段化目标、经验、硬约束、风险偏好、资源和披露边界。这是信息最清楚的条件，用来测“给到之后会不会用”。

3.2 Natural history/context

同样的信息藏在自然叙述、历史对话、文档或选择记录里。它和 structured persona 可以做语义等价检查：意思相同，系统关键决定应该基本一致。

3.3 Fuzzy query + clarification

初始 query 不是无法执行，而是足以写一份普通报告、却缺少 1-3 个会改变建议的用户条件。agent 可以直接做，也可以先问。用户模拟器只能按 ledger 回答；超出 ledger 的问题回答 unknown，避免它临场编造 persona。

这个条件额外记录：

- 该问的节点有没有发现；
- 问题是否精准、有没有一次问太多；
- 得到回答后是否进入计划和最终建议；
- 是否问了本可自己检索的问题；
- 是否询问不必要的敏感信息；
- 交互轮数和用户负担。

3.4 Task-only

不给任何任务相关用户信息，但仍要求写一份高质量通用报告。它不是故意做差的 baseline，而是用来判断 matched 是否真的新增了价值。

3.5 第一批数据已经开始

现在有 3 个纯合成的工程 family：团队知识平台选型、七天日本家庭旅行、文献综述工作流选型。每个 family 有两位约束不同但都自然的用户，每位用户生成 P0/P1/P2/P4 四个 episode，共 24 个。程序已经检查每种范式各 6 个、fact ID 不乱引用、P2 确实隐藏决策关键信息、P4 确实只有一个覆盖旧事实的更新。

它们现在只能回答“数据结构和实验流程能不能跑”，不能回答“真实用户中的 agent 表现”。下一步必须人工审核三个任务是否自然、证据包是否足以决定、A/B matched 与 swapped 是否真的能稳定区分，再用真实或真人锚定的 seed 替换或验证。

我们也已经把 PDR-Bench 公开的全部资源导入：50 个任务、25 个志愿者自填后去标识化的 structured persona、25 份由专业标注者模拟的 dynamic context，以及 250 个官方 task-user 配对。这里必须说准确：persona 的结构化底稿有真人来源，但 context 不是志愿者自然聊天轨迹。

这 250 对不能直接当成 DeepAlign 的 250 个独立实验。v0.54 把 PDR 放进更大的候选池：总共先收 180 个 seed（72 研究、54 软件、54 数据），再预选 60 个 family（24/18/18）。其中 12 个直接来自 PDR task shell，其他补文献景观、prior art、仓库修改、spreadsheet 和 ML 等结构。60 个只是 provisional sampling frame，没有通过许可、环境、双人审查、contract 和 pilot 门就不能说是可运行 gold。

4. 输出条件怎么构造

对每个 family 至少需要：

- y_0 ：task-only 通用报告；
- y_a ：为用户 A 生成的报告；
- y_b ：为用户 B 生成的报告。

两套用户标准都要评价 y_a 、 y_b 和 y_0 。不能先看到 swapped 报告，再专门给它添加扣分项。

D-JQS 还要有四种反例：

- general-good：整体质量很好，但没有做到 A/B 必须不同的地方；
- over-personalized：大量提到用户信息，却在一个关键决定上采用了错误约束；
- mention-only：报告写到了某个约束，但最后并没有真的按它选方案；
- irrelevant-keyword：加入显眼的人口属性或 persona 词，实际不应改变建议。

5. Rubric 为什么要提前设计，judge 为什么要先考试

Rubric Compiler 的输入不是“让 LLM 随便想评分点”，而是 task metadata + 授权 ledger 引用 + 已冻结 CDM。

它依次做：

metadata/CDM → module → direction node → parameterized leaf → 人工校验并冻结

- Module 是大方向，例如事实正确性、风险适配、行动可行性、隐私边界。
- CDM node 先决定“A/B 到底应该有什么关系”；direction node 只提供可复用的测量写法。
- Leaf 是这个 case 里的具体可判定项，例如“最终推荐不得把只支持 SaaS 的方案作为敏感项目默认方案”。

每个 leaf 要写清它来自哪个 CDM node、适用对象、证据位置、评分锚点、严重性、是否 hard gate、和其他 leaf 是否共享同一个 parent，以及它评价的是 mention、planning 还是 final adoption。多个 leaf 来自同一节点时先在节点内合并，不能把一个偏好拆成五条就获得五倍权重，也不能把五条当成五个独立样本。

能用程序检查的先用程序：code 的 tests、data 的数字/公式、DR 的链接和部分 claim support。但程序 checker 也可能漏错，所以要用已知正例/反例、受控修改和 mutation test 估计 false accept、false reject 与覆盖率。“确定性”不等于“天然正确”。

只有 trade-off、优先级、actionability 等语义项交给 LLM judge。judge 只应用已经冻结的 leaf，并必须指出 artifact 里的证据位置；不能再发明新标准。项目内校准工具改名 **D-JQS (DeepAlign Judge Qualification Suite)**，因为已有论文已经叫 JudgeBench/JUDGE-BENCH。[\[6\]\[7\]](#) D-JQS 同时用明确违规、只改一个关键点的受控 artifact、以及真实用户盲选三种 gold；调阈值集和隐藏资格集按任务、用户、agent、来源和时间隔离。RuVerBench 已直接表明长 agentic artifact 的 rubric verification 仍有明显噪声；GAMUT 又已覆盖 two-level meta-rubric 编译，因此这两项不能被我们误报为首创。[\[8\]\[9\]](#)

AB/BA 只能测位置偏差，所以还要单独改长度、文风、格式、persona 关键词、引用数和语言。某个 judge 只在事实判断上合格，不代表它能判断用户取舍；资格按 leaf 类型给。关键类型没过门就交给别人，不能让几个都不可靠的 judge 投票后假装可靠。

5.1 这套方案最容易被怎么打

- “你只挑最好分的用户对。” 公开 offered→selected→paired→rejected 漏斗，并同时报告 contrast、near-neighbor、neutral pair。
- “用户自述会变。” 允许不确定和多种可接受答案，做 test-retest，给事实加时间戳/到期复核；不稳定方向不进 gold。
- “用户前面定标准，后面自然会选 matched。” 后期不展示最终 rubric，随机顺序、隐藏条件，尽量分时进行；用户盲选是外部效度，不是同一 gold 的重复证明。
- “CDM 还是不完整。” 只声称在预注册访谈和 coverage audit 下达到有限 saturation；测试输出后发现的新错误不回改主分。
- “D-JQS 自己给自己发证。” gold 三来源、calibration/hidden split、按 slice 认证；失败 slice 必须人工接管。
- “这只是 PDR++ 或约束遵循。” 比较 PDR-style 单用户 rubric、独立 A/B rubric 和 CDM 对称 rubric，并证明 CDM 会重分类系统，或在控制一般质量后更能预测真人选择；persona 同时覆盖硬约束、真实取舍、知识/受众和需要澄清的隐含需求。

- “三个领域根本不可比。” 不比较跨领域 raw score；分别报告 code/data/DR 的 verifier 覆盖和 judge 负担，只统一 matched/swapped 与 no-harm 逻辑。
- “太贵、隐私风险高。” 先做 12-family paper set；raw ledger 默认不发布，必须有 consent、最小化、撤回、保留期限和访问控制。

6. 分数到底怎么算，为什么不是又一个差值总分

令 $PF_a(Ya)$ 表示 A 的标准给 A 报告的分数，其他类似。

- $\Delta a = PF_a(Ya) - PF_a(Yb)$ ：对 A 来说，A 报告是否真的优于 B 报告。
- $\Delta b = PF_b(Yb) - PF_b(Ya)$ ：对 B 来说，B 报告是否真的优于 A 报告。
- $CFA_min = \min(\Delta a, \Delta b)$ ：只要一个方向失败，整体就不能说双向成功。
- $A_min = \min(PF_a(Ya), PF_b(Yb))$ ：防止 matched 本身只有 4 分，但 swapped 只有 0 分，于是差值显得很大。
- $Ga = PF_a(Ya) - PF_a(Y0)$ 、 $Gb = PF_b(Yb) - PF_b(Y0)$ 、 $Gain_min = \min(Ga, Gb)$ ：防止只因为 swapped 特别差就宣称个性化有收益。

差值本身没有错。实验想知道“只改变用户条件造成了什么效果”，本来就需要受控差值。错误的是把一个差值当作所有问题的答案。

所以最终不输出单一 personalization 总分，而输出一个 profile：

1. 双向 specificity 是否过线；
2. matched 的绝对适配是否过线；
3. 相对 task-only 是否至少不差、是否有真实新增收益；
4. 共同任务质量和事实可靠性是否没有下降；
5. critical must-not、隐私和权限是否零严重违规。

PF leaf 先统一到 $[0, 1]$ ，因此 Δ 已经是量尺范围归一化的百分点差。再除以 matched+swapped 会让低分区的小差异变得特别大。向量夹角可以诊断两边方向是否一致，但 $0.01/0.01$ 也会得到完美方向，所以仍不能代替幅度和绝对门槛。

7. 统计为什么按 task family 聚类

同一个 family 里的 A/B 用户、多个 seed、多个 judge 重复和很多 rubric leaf 都共享任务和证据，因此不是独立样本。

- permutation test：在每个 family 内交换 matched/swapped 条件标签，问“如果条件本来没有效应，这么大的配对差异有多常见”；
- cluster bootstrap：每次抽取完整 family，而不是随便抽一条分数，保持 family 内相关结构；
- 最终置信区间按 family 聚类，不能把 20 个 leaf 当 20 道独立题来缩小误差。

8. 这次最小实验结论

我们运行了两个合成 family、两位用户，以及 general-good、matched、swapped 和 over-personalized artifacts。评分使用 PDR-compatible 四维构念和动态 criteria，但 judge 是本地 Qwen3-8B，不是官方 GPT-5。

结果：

- general-good 4/4 次都在 matched 的 0.5 分内，其中一次还高于 matched；
- over-personalized 4/4 次都高于 6 分，但只有 1/4 次在 matched 的 0.5 分内；
- 两个 family 的 matched 绝对分都很高，但 CFA_min 一组为 -1.50、一组为 0；
- 出现了“表格里提到本地部署，就被当成最终建议已采用”的 mention-adoption 错误；
- 另一组多个不同报告全部 10 分，显示高分饱和。

人话结论是：通用好报告很可能被绝对个性化分评得和 **matched** 一样好；过度个性化报告并没有普遍骗过 **judge**，但关键错误有时会被平均分补偿或完全漏掉。所以 DeepAlign 的方向值得继续，但现在还不能对外说官方 PDR-Bench 已经失效。

我们随后把 GPT-5 复现做成了真正的“先锁题再看结果”：先把 4 个 family、20 份报告、官方 PDR 中文 prompt、5 次权重采样、A/B 全交叉和三次重复提交到 Git，再调用 API。结果不是 GPT-5 给了零分或高分，而是 **根本没有进入 GPT-5**。OpenRouter 确认 key 有效、有余额、GPT-5 也在可用模型列表里，但根据账户/请求地区的 provider terms，在选择 OpenAI/Azure endpoint 前就返回 403。去掉隐私筛选也一样。因此目前没有新的 GPT-5 实验结论，不能把这个工程阻塞写成支持或反对论文假设的证据。

换句话说：本地最小实验结论没有被 GPT-5 复现，也没有被 GPT-5 推翻。现在需要的是一个在受支持账户/地区合法可用的 GPT-5 key，或者官方 OpenAI API key；不应该用代理或别的模型冒充复现。

还要特别防止一种说法升级：如果 GPT-5 也给通用好报告高分，这不等于 PDR-Bench “打错了”。通用报告可能真的对这位用户有帮助，所以绝对适配分高是合理的。它只能说明这个分数回答不了“系统有没有因为换了用户而改变”。真正更强的反例必须同时满足：人类不知道报告类型时仍一致认定最终决定违反了关键用户约束；GPT-5 三次评分却仍接近 **matched** 或超过 **matched**；而这种分歧不只发生在一个 family。再进一步，只有它会改变多个 agent 的成功判定或排名，并且 DeepAlign 更贴近真实用户判断，才够支撑论文最重要的测量效度贡献。

因此我预期“通用好报告接近 **matched**”较容易复现；“关键决策错误的 over-personalized 报告普遍接近 **matched**”不确定，现有本地结果反而只有 1/4。后者如果没有出现，不是实验失败，而是明确否决强 PDR 缺陷假设，论文就只保留 absolute fit 与 counterfactual specificity 的构念差异。

9. 接下来五天必须完成什么

ICLR 2027 官网当前给出的摘要截止是 2026-09-18 AOE、全文截止是 2026-09-25 AOE。从 8 月 14 日起约有 35/42 天。过去记录中的 9 月 11/16 已被官网更新，不能再沿用。

因此我建议最迟 8 月 17 日冻结方向。不是五天后任何细节都不能改，而是五天后不能再改论文究竟测什么。必须冻结：

- 论文 thesis：absolute adaptation 不等于 counterfactual specificity；
- 主数据单位：paired-user task family；
- 主结果：非补偿 profile，而不是总分；
- 核心反例：general-good、over-personalized、mention-only；
- 最近邻边界：承认 PDR、MyScholarQA、G-STEER 等已经覆盖的部分。

当前 go/no-go：先把三个 vertical 各 1 个 family 做出真人 ledger、CDM、受约束 leaf、validated verifier 和 D-JQS/human 评分；至少 2/3 family 的 reference matched 必须稳定优于 swapped，neutral pair 不能被无故改坏。若 CDM 相对独立 A/B rubric 不产生任何系统重分类，也不增量预测真人选择，就把论文降级为透明 measurement extension，不把 compiler 包装成核心创新。

之后先在三个 vertical 各完成一个端到端环境，再把主论文补到 12 个 family（5 研究 / 3 软件 / 4 数据）。两个月内不承诺把 60 个全部变成 runnable case；如果资源不足，先减 agent 数和动态压力层，不把 software/data 又降回几个展示 anchor。

10. 论文最后可以声称什么

如果主实验通过，可以声称：DeepAlign-Bench 提供了固定 task/evidence/resources 下的 paired-user 反事实评测；它用真人来源的 relational CDM 约束 rubric，用 D-JQS/hybrid scoring 执行标准，

并能把通用高质量、单边个性化、低绝对适配、只胜过差 swapped、关键约束误用和边界违规分开。

不能声称：模型真正理解用户；clarification 是首次提出；所有高分 PDR 报告都是假阳性；或 artifact specificity 自动等于真实用户收益。

参考文献

- [1] [Towards Personalized Deep Research: Benchmarks and Evaluations](#). ICLR 2026.
- [2] [Language Models Don't Know What You Want / MyScholarQA](#). ACL 2026.
- [3] [IDRBench](#). 2026.
- [4] [G-STEER](#). 2026.
- [5] [ICLR 2027 Author Guidelines](#). 2026.
- [6] [JudgeBench](#). ICLR 2025.
- [7] [JUDGE-BENCH](#). ACL 2025.
- [8] [RuVerBench](#). 2026.
- [9] [GAMUT](#). 2026.